

Using the HMRC Self Assessment (SA) Client

Date 09/02/2021

Author Ian H

Version 1.0

Introduction

This document covers the SA Client component and the SA demo app from the UKDG attempt to handle the HMRC MTD APIs. The process started towards the end of 2018, with a coding session at one of the meetings in early 2019 followed by the release of the first version of the API client, then handling only the VAT and test APIs, as they were at that time. The first update was released in December 2020, nearly 2 years after the start, with another update in January 2021, including the updated anti-fraud headers. This document accompanies the third update released in early February 2021, which includes the first release of the Self Assessment (SA) client.

It is assumed that the reader is familiar with the HMRC MTD process, so understands the authorisation access levels and the formats of the endpoints which they wish to access. The general or core tests should be relevant to everyone wishing to set up tests in the sandbox. Other tests will be relevant only to those accessing that particular API. The stand alone test project now includes both the original test demo form and processes, but also the SA setup processes, both of which will be required to create test users in the sandbox and create test data for the SA endpoints. Read TheTestDemos.pdf for details. The original documentation, then called UsingTheDemos, but now renamed UsingTheVatDemos, would probably also be of interest and/or benefit, because it covers the base rest client and some of the core processes, as well as some vat specific stuff. It was written for the original release when there was only VAT.

The SA client component gives the developer the option of accessing the data returned from the endpoint as json, but also in other formats, removing the need to either read / parse json or to create / format json, if the developer would prefer not to. The SA demos all use the data objects, defined in the HmrcSASupport unit, so there is no json in the actual forms.

The SA Client

```
THmrcSAClient = Class(THMRCRestClient)
protected
function IzValidId: boolean; override; // does the UID look valid
public
constructor Create(AOwner: TComponent); override;
// get the individual benefits
function GetBenefits(Const AValue: string): integer; overload;
function GetBenefits(Const AValue: string; var ACode: integer): TSABenefitsList; overload;
// get the individual employments
function GetEmployment(Const AValue: string): integer; overload;
function GetEmployment(Const AValue: string; var ACode: integer): TSAEmploymentsList;
overload;
// get the individual income
function GetIncome(Const AValue: string): integer; overload;
function GetIncome(Const AValue: string; var ACode: integer): TSAIncomes; overload;
// get the marriage allowance eligibility
function GetMAEligibility(AValue: TSAMrgAllowance): integer;
```

```

// get the marriage allowance status
function GetMAStatus(const AValue: string): integer; overload;
function GetMAStatus(const AValue: string; var AStatus: string; var ADead: boolean): integer;
overload;
// get the national insurance
function GetNI(Const AValue: string): integer; overload;
function GetNI(Const AValue: string; var ACode: integer): TSANI; overload;
// get the individual tax
function GetTax(Const AValue: string): integer; overload;
function GetTax(Const AValue: string; var ACode: integer): TSAIndividualTax; overload;
// override to apply some kind of validation
Function SetHmrcID(Const Value: String): integer; Override;
end;

```

There is an internal function to perform some validation of the UID, which in this case is a UTR. There is also an overridden version of the SetHmrcId function which also provides some validation of the value supplied. For SA/PAYE/NI, the UID is usually the UTR, but occasionally the NINo. Why is it called UID and not UTR? On the VAT side, it would be the VRN (VAT Registration No), so it is not a single value type.

Apart from those, there are 2 functions to access 6 of the 7 endpoints and only 1 function for the other one. The solo function and one of each of the other pairs return a status value indicating either success or failure. If successful, then the data returned from the endpoints can be accessed as LastValue, in the form of a TJSONValue. This will require System.JSON to be declared in the uses clause.

The other functions mainly return an appropriate data object, removing the need to handle json. but the Marriage Allowance (MA) functions are a bit different from the others. Most functions also just accept a tax year either as the single parameter, or as the first parameter. Except for the MA functions, the second parameter, as a var parameter, will be a call result status code. The MA eligibility function, however, only accepts a marriage allowance data object as the input parameter. This is defined in the HmrcSASupport unit, along with the other SA related data objects and methods to parse the json into the objects. A usage example of each of the methods that return data objects is shown in the demo project and some are explained below.

A note for traditional Delphi database programmers who are new to REST and the internet way of working, where normally a call to load data would just return an empty dataset, if nothing was found, with the API, no data results in a 404 Not Found response. No data is treated the same as an invalid request or a processing error.

Declaring and creating a SA component in code.

Uses

```

..., HmrcRestClient, HmrcRestSupport, HmrcSAClient, HmrcSASupport, ...
type
  TSAForm = class(TForm)      // your application form
...
  SAClient : THmrcSAClient;
...
public
  procedure SaveToken(Sender: TObject; const aVRN, aScope, aToken, aRefresh: string;
    const aExpiry, aTimeOut: TDateTime);

```

```

procedure LoadDetails;           // load saved application details
procedure LoadTokens(const Value: string); // load any stored access tokens for the current user
end;

```

In the FormCreate method, create the SA client component and set token saving method. Also load the client keys, here represented by the LoadDetails method and optionally also load all stored access tokens. Tokens may be loaded for individual users later, if that is preferred.

```

procedure TSAForm.FormCreate(Sender: TObject);
begin
    SAClient := THmrcSAClient.Create(Self);

    SAClient.OnTokenChange := SaveToken; // this should automate token saving

    // optionally set token load procedure – remove this assignment if not required
    SAClient.DoLoadTokens := LoadTokens;

    LoadDetails;
    // optionally load all tokens now – remove this call if loading on demand
    LoadTokens("");
end;

```

The SaveToken method should match the signature of the THmrcTokenEvent type, declared in HmrcRestSupport. This should save the token details to the chosen data store. The demo projects use an ini file, because it is the simplest to demonstrate, but it can be any storage mechanism. The SAClient component will automatically save new and updated access tokens using this process, if it is set. Otherwise saving tokens would have to be done by some other process. It is assumed that the reader understands OAuth2 and access tokens.

Similarly, saved access tokens need to be loaded by a form/application process and passed to the SAClient. The SAClient provides 2 methods for accepting tokens, declared in the parent RestClient class:

```

Procedure AddToken(Const uid, scp, atn, rtn: String; Const exp, tmo: TDateTime);
Procedure SetaToken(Const scp, uid, atn, rtn: String; Const exp, tmo: TDateTime);

```

The load tokens method should match the signature of the THmrcTokenLoad type, declared in HmrcRestSupport, if it is to be used by the client to load dynamically on demand. If an empty string is passed as the parameter, it should load all saved tokens, otherwise it should load only tokens for the supplied UID. In the SA test demo it loads tokens on demand, to show how this should work, but since this is something for the application developers to write, it can do whatever they want. It can either be pro-active, where the form triggers the loading, when the UID (UTR) is set, with a form event or it can be reactive and called from the client when needed, although this requires it to be set for the DoLoadTokens event.

In the demo main form, it also initializes some data for the anti-fraud headers in the FormCreate method. This is the data that needs to be set manually, but does not necessarily need to be set in this way. See the section below on the headers for more detail.

The SA client methods accept parameters from the host application and format them, along with other fixed values for that endpoint, to make a call to the almost universal CallApi method of the base rest client class. This document will not look at that process, only to say that it handles all of

the tokens and returns a status value indicating success or failure. If successful, the object based method calls then convert the json returned into data objects.

To use the SA client component, it is not necessary to understand how it works internally, only to know what it needs to be told. Below are example calls taken from the SA demo app.

The main form of the SA client demo just has one option, to open the client form, which is shown below. It has one button for testing each of the 7 endpoints and where each requires the UTR and the appropriate tax year to be set. The UTR entered in the edit is set as the client UID in the OnExit method, which simplifies the process. It also means that it does not have to be passed as a parameter. The MA Eligibility endpoint also requires the additional data identified on the form. This is also documented in the HMRC API web docs.

The screenshot shows a software application titled "Test SA Client". It features a yellow header with the text "UKDG". Below the header, there are two input fields: "UID (UTR)" containing the text "UTR" and a dropdown menu for the tax year, currently set to "2018-19". Below these are seven buttons arranged in a grid: "Benefits", "Employments", "Income", "MA Status", "National Insurance", "Tax", and "MA Eligibility". The "Benefits" and "National Insurance" buttons are highlighted with blue borders. At the bottom of the form, there is a section for "MA Eligibility" with four input fields: "Firstname" (value "firstname"), "Surname" (value "surname"), "NINo" (value "NINo"), and "DOB" (value "YYYY-MM-DD"). A large empty rectangular box is at the very bottom of the form.

The Benefits call using benefit data objects.

```
procedure TfrmTestSA.DoBenefits;
var
  errcode: integer;
  Benefit: TSABenefits;
  Benefits: TSABenefitsList;
begin
  Benefits := OTestClient.GetBenefits(cbxYear.Text, errcode);
  if (Assigned(Benefits)) then
  try
    // succeeded, so write data returned to display
```

```

mmoInfo.Lines.Add("");
mmoInfo.Lines.Add('Benefits');
for Benefit in Benefits.Benefits do
begin
    mmoInfo.Lines.Add("");
    mmoInfo.Lines.Add('PayeRef : ' + Benefit.PayeRef);
    mmoInfo.Lines.Add('Car benefit : ' + IntToStr(Benefit.CCVBens));
    // ... etc...
end;
// or do something more useful with it
// ...
finally
    Benefits.Free;
end // if success
else
    DoError(errcode);
end;

```

If successful, the benefits endpoint will return a list of benefits, as defined in the HMRC docs. The GetBenefits call, with the tax year and status place holder as parameters, will return either nil or a benefits list object. The presence of the object indicates success, so the object can then be used as required. In the demo, it is just written to the display, because there is nothing else to do with it. If no object is returned, then it calls the error handling routine with the status value (errcode). Simple.

Employments, Income, National Insurance and Tax all work the same way, but with appropriate data objects. The data objects are defined in the HmrcSASupport unit. The common error handling method just checks the error value and writes a corresponding message to the display, but could do something more useful. Note that exceptions raised in the call will be swallowed and returned as an error, with value RESULT_ERROR. Failure for other reasons will have a value of RESULT_FAIL.

The common error handling method.

```

procedure TfrmTestSA.DoError(const AValue: integer);
begin
    // check whether it failed because there was no access token
    if (OTestClient.LastCode = ERR_NO_ACCESS_TOKEN) then
    begin
        // so get a new access token
        OTestClient.NewAccessToken;
        // now the user needs to call the api again
        mmoInfo.Lines.Add('Please call the API again, after authority has been granted');
    end // else no token

    // handle error situation
    else begin
        if (AValue = RESULT_NONE) then
            // failed, but should only be because the UID is invalid
            mmoInfo.Lines.Add('No UID (UTR) or UID is invalid')
        else if (AValue = RESULT_FAIL) then
            // general failure, so write response info to display
            mmoInfo.Lines.Add('Called failed with: ' + IntToStr(OTestClient.LastCode) + ' ' +
OTestClient.LastMsg)

```

```

else
    // exception, so write error info to display
    mmoInfo.Lines.Add('Exception: ' + IntToStr(OTestClient.LastCode) + ' ' +
OTestClient.LastMsg);

    // and write the error/exception message
    mmoInfo.Lines.Add(OTestClient.LastError);
    // or handle it some other way ...
end; // else failure
end;

```

In the case of no access token, the error handler does call the NewAccessToken method of the SAClient. This will pop up the authorisation form and then get a new access token, following successful authentication. The original button will need to be clicked again to restart the process.

The two Marriage Allowance methods are slightly different from the other 5. Status simply passes the tax year as the input parameter and 2 var parameters for the returned data, 1 string and 1 boolean. As with the others, the result is just written to the memo.

```

procedure TfrmTestSA.DoMAStatus;
var
    errcode: integer;
    AStatus: string;
    IzDed: boolean;
begin
    errcode := OTestClient.GetMAStatus(cbxYear.Text, AStatus, IzDed);
    // check the status
    if (errcode = RESULT_OK) then
    begin
        // succeeded, so write data returned to display
        mmoInfo.Lines.Add("");
        mmoInfo.Lines.Add('MA Status');
        mmoInfo.Lines.Add(' Status : ' + AStatus);
        mmoInfo.Lines.Add(' Deceased : ' + BoolToStr(IzDed));
        // or do something more useful with it
        // ...
    end // if success
    else
        DoError(errcode);
end;

```

The status and deceased values were extracted from the json returned by the endpoint. It really cannot get much simpler than that. This endpoint differs from the others in that the tax year has to be submitted as a parameter rather than as part of the url, so the call handler has to deal with this internally.

The second MA endpoint only returns 1 boolean value, but requires a json object to be submitted in the body of the request, although there are no parameters. The method below starts by declaring a data object and setting the values from the data entered into the form. The SA client then accepts this object as the single parameter, from which it gets a json string to submit to the endpoint. The response, if successful, is a single boolean value, which is used to update the data object.

```

procedure TfrmTestSA.DoMAEligibility;
var
  errcode: integer;
  Mae: TSAMrgAllowance;
begin
  // initialize the data object
  Mae := TSAMrgAllowance.Create;
  try
    Mae.UTR      := OTestClient.UID;
    Mae.TaxYear  := cbxYear.Text;
    Mae.Nino     := Trim(edtNino.Text);
    Mae.Firstname := Trim(edtFname.Text);
    Mae.Surname  := Trim(edtSname.Text);
    Mae.DOB      := Trim(edtDOB.Text);

    errcode := OTestClient.GetMAEligibility(Mae);
    if (errcode = RESULT_OK) then
    begin
      // succeeded, so write data returned to display
      mmoInfo.Lines.Add("");
      mmoInfo.Lines.Add('MA Eligibility');
      mmoInfo.Lines.Add(' Eligible : ' + BoolToStr(Mae.Eligible));
      // or do something more useful with it
    end // if success
  else begin
    DoError(errcode);
  end;
finally
  Mae.Free;
end;
end;

```

The anti-fraud headers are covered in more detail elsewhere, but briefly they are supposed to be required for all APIs at some point, so it is better to get used to them now. At the time of writing, the package contains a set of support units to help with setting the headers, but only the desktop version. It includes 2 processes for adding the headers, one in HmrcHeaders and the other in Systematic.VAT.Headers. The demo uses the version from the HmrcHeaders unit, but the application developers can use either of these or roll their own.

Some of the header units are or use FMX units. These have been tested in the VCL demos and have been found to work without any issues, although there may be a compiler warning. Similarly, the base rest client contains 1 compiler switch based on a conditional define of FMX, which should be set in FMX projects. This is because the authorisation form is framework specific. In testing, however, the FMX demo worked with the VCL form and the VCL demo worked with the FMX form, but to avoid compiler warnings and potential problems, the conditional define should be used in FMX projects.

Some Links

The tutorials were here

<https://developer.service.hmrc.gov.uk/api-documentation/docs/tutorials>

The API listing was here

<https://developer.service.hmrc.gov.uk/api-documentation/docs/api/>

The desktop fraud headers were here

<https://developer.service.hmrc.gov.uk/guides/fraud-prevention/connection-method/desktop-app-direct/>